Ingenieria de software 1

Jaime Nazar Anchorena

Contents

Capitulo 1: Software y la ingenieria de software **3**
Naturaleza del software . . . 3
Definicion . . . 3
Caracteristicas que difieren del herdware . . . 3
Categorias . . . 3
Software heredado . . . 3
Naturaleza unica de las webapps . . . 4
Ingenieria de software . . . 4
Definicion . . . 4
Proceso del software . . . 5
Practica de la ingenieria de software . . . 5
Escencia de la practica . . . 5
Principios generales . . . 6

Capitulo 2: Modelos del proceso **6**
Modelo general de proceso . . . 6
Definicion: Actividad estructural . . . 6
Identificacion de un conjunto de tareas . . . 8
Patrones del proceso . . . 8
Evaluacion y mejora del proyecto . . . 8
Modelos del proceso prescriptivo . . . 9
Modelo de procesos especializado . . . 9
Desarrollo basado en componentes . . . 9
Modelo de metodos formales . . . 9
Desarrollo orientado a aspectos . . . 10
El proceso unificado . . . 10
Fases del proceso unificado . . . 10
Modelos del proceso personal y del equipo . . . 10
Proceso personal del software(PPS) . . . 12
Proceso del equipo de software(PES) . . . 12
Tecnologia del proceso . . . 12
TL;DR . . . 13

Capitulo 3: Desarrollo agil **13**

Manifesto agil 13
Que es la agilidad? 13
La agilidad y el costo del cambio 14
Que es un proceso agil? 15
Principios de agilidad 15
La politica de desarrollo agil 15
Factores humanos 16
Programacion extrema(XP) 16
Valores 16
Proceso 16

Capitulo 1: Software y la ingenieria de software

Naturaleza del software

Definicion

El software es:

1. Instrucciones (programas de cómputo) que cuando se ejecutan proporcionan lascaracterísticas, función y desempeño buscados.
2. Estructuras de datos que permiten que los programas manipulen en forma adecuada la información.
3. Información descriptiva tanto en papel comoen formas virtuales que describen la operación y uso de los programas.

Caracteristicas que difieren del herdware

- El software se desarrolla o modifica con intelecto; no se manufactura en el sentido clásico.
- El software no se "desgasta".
- Aunque la industria se mueve hacia la construcción basada en componentes, la mayor parte del software se construye para un uso individualizado.

Categorias

Hoy en dia hay siete grandes categorias:

- Software de sistemas
- Software de aplicacion
- Software de ingenierias y ciencias
- Software incrustado
- Software de linea de productos
- Aplicaciones web
- Software de inteligencia artificial

En ciertos casos se elabora software nuevo, pero muchas veces simplemente se corrige, adapta y mejora aplicaciones ya exitentes. En base a eso, aparecieron nuevos desafios:

- Computacion en un mundo abierto(computacion distribuida)
- Construccion de redes(poder crear arquitecturas sensillas para algo tan complejo)
- Fuente abierta(el codigo debe ser autodescriptivo para que sea mas sensillo colaborar)

Software heredado

Se denomina **software heredado** a los programas mas antiguos que siguen siendo utilizados, son costosos de mantener y muy dificiles de hacerlo evolucionar.

Este tipo presenta caracteristicas como las mencionadas anteriormente y tambien se le agrega que en general es de mala calidad.

- No facil de actualizar
- Codigo confuso
- Documentacion mala o inexistente
- Casos y pruebas que nunca se archivaron

- Historial de cambios mal administrada

Podria dejarse como esta y seguira funcionando, pero puede ser necesario actualizarlo si:

- El software debe adaptarse para que cumpla las necesidades de los nuevos ambientes del cómputo y de la tecnología.
- El software debe ser mejorado para implementar nuevos requerimientos del negocio.
- El software debe ampliarse para que sea operable con otros sistemas o bases de datos mmodernos.
- La arquitectura del software debe rediseñarse para hacerla viable dentro de un ambiente de redes.

Naturaleza unica de las webapps

La mayoria presentan los siguientes atributos:

- Uso intensivo de redes
- Concurrencia
- Rendimiento
- Disponibilidad
- Orientada a los datos
- Contenido sensible
- Evolucion continue(por ejemplo, youtube, estado del subte)
- Inmediatez(Llegan rapidamente al mercado)
- Seguridad
- Estetica

Ingenieria de software

- El software esta en todos los aspectos de nuestras vidas, muchos clientes. Se concluye que debehacerse un esfuerzo concertado para entender el problema antes de desarrollar una aplicación de software.
- Las demandas son cada vez mas complejas. Se concluye que el diseño se ha vuelto una actividad crucial.
- Se depende cada vez mas del software para tomar decisiones. Se concluye que el software debe tener alta calidad.
- Al aumentar su valor percibido, aumenta la cantidad de usuarios. Se concluye que el software debe tener facilidad para recibir mantenimiento.

Definicion

a ingeniería de software es una tecnología con varias capas. Cualquier enfoque de ingeniería (incluso la de software) debe basarse en un compromiso organizacional con la calidad. La administración total de la calidad, Six Sigma y otras filosofías similares 10 alimentan la cultura de mejora continua, y es esta cultura la que lleva en última instancia al desarrollo de enfoques cada vez más eficaces de la ingeniería de software. El fundamento en el que se apoya la ingeniería de software es el compromiso con la calidad.

- **Proceso**: El proceso de software forma la base para el control de la administración de proyectos de software, y establece el contexto en el que se aplican métodos técnicos, se generan productos del trabajo (modelos, documentos, datos, reportes, formatos, etc.), se establecen puntos de referencia, se asegura la calidad y se adminis- tra el cambio de manera apropiada.

- **Metodos**: Los métodos de la ingeniería de software se basan en un conjunto de principios fundamentales que gobiernan cada área de la tecnología e incluyen actividades de modelación y otras técnicas descriptivas.
- **Herramientas**: Proporcionan un apoyo automatizado o semiautomatizado para el proceso y los métodos. Cuando se integran las herramientas de modo que la información creada por una pueda ser utilizada por otra, queda establecido un sistema llamado ingeniería de software asistido por computadora que apoya el desarrollo de software.

Proceso del software

Conjunto de actividades, acciones y tareas que se ejecutan al crear algun producto:

- **Actividad**: Busca lograr un objetivo amplio.
- **Accion**: Conjunto de tareas que producen un producto(diseno de arquitectura).
- **Tarea**: Objeto pequeno, pero bien definido(prueba unitaria), produce resultado tangible.

El proceso no es una receta a seguir, sino que se adapta al equipo de trabajo. La estructura del proceso esta formada por las siguientes **actividades estructurales**:

- Comunicacion(con el cliente)
- Planeacion
- Modelado(para entender mejor los requerimientos)
- Construccion
- Despliegue(entrega al consumidor)

Estan son la base, sirven para proyectos sencillos, pero para software mas grande se vuelve mas complejo.

Las actividades estructurales del proceso de ingeniería de software son complementadas por cierto número de **actividades sombrilla**. Estas son:

- Seguimiento y control del proyecto de software
- Administración del riesgo
- Aseguramiento de la calidad del software
- Revisiones técnicas
- Medicion
- Administración de la configuración del software
- Administración de la reutilización
- Preparación y producción del producto del trabajo

  Nota: Un proceso adoptado para un proyecto puede ser significativamente distinto de otro adoptado para otro proyecto.

Practica de la ingenieria de software

Escencia de la practica

Para poder resolver un problema, Polya plantea lo siguiente:

1. Entender el problema (comunicación y análisis).
2. Planear la solución (modelado y diseño del software).
3. Ejecutar el plan (generación del código).

4. Examinar la exactitud del resultado (probar y asegurar la calidad).

Principios generales

Estos ayudan a establecer herramientas mentales para una practica solido. Hooker porpone siete principios:

1. Lo razon de que exista todo, debe dar valor a sus usuarios.
2. KISS(Keep It Simple, Stupid), MSE en castellano.
3. Mantener la vision, una visión clara es esencial para el éxito de un proyecto de software.
4. Otros consumiran lo que usted produce, siempre establezca especificaciones, diseñe e implemente con la seguridad de que alguien más tendrá que entender lo que usted haga.
5. Abrase al futuro
6. Plantee por anticipado la reutilizacion, no reinventar la rueda
7. Piense! Pensar en todo con claridad antes de emprender la acción casi siempre produce mejores resultados.

   Nota: Si todo ingeniero y equipo de software tan sólo siguiera los siete principios de Hooker, se elimi narían muchas de las dificultades que se experimentan al construir sistemas complejos basados en computadora.

Capitulo 2: Modelos del proceso

Se define proceso del software como una estructura para las actividades, acciones y tareas que se requieren a fin de construir software de alta calidad.

Modelo general de proceso

Dentros de las actividades sobrilla se realizan las actividades estructurales, y el conjunto de actividades sobrilla forman la estructura del proceso. Sin embargo, falta mencionar el flujo, escribe la manera en que están organizadas las actividades estructurales y las acciones y tareas que ocurren dentro de cada una con respecto de la secuencia y el tiempo.

Hay varias maneras de realizar el flujo:

- Un **flujo de proceso lineal** realiza las cico actividades estructurales en secuencia.
- Un **flujo de proceso evolutivo** realiza las actividades en forma circular.
- Un **flujo de proceso paralelo** ejecuta una o mas actividades en paralelo.

Definicion: Actividad estructural

¿qué acciones son apropiadas para una actividad estructural, dados la naturaleza del problema por resolver, las características de las personas que hacen el tra bajo y los participantes que patrocinan el proyecto?

Por ejemplo: Para un proyecto sencillo:

1. Hacer contacto con el participante por vía telefónica.
2. Analizar los requerimientos y tomar notas.
3. Organizar las notas por escrito en una formulación breve de los requerimientos.
4. Enviar correo electrónico al participante para que revise y apruebe.

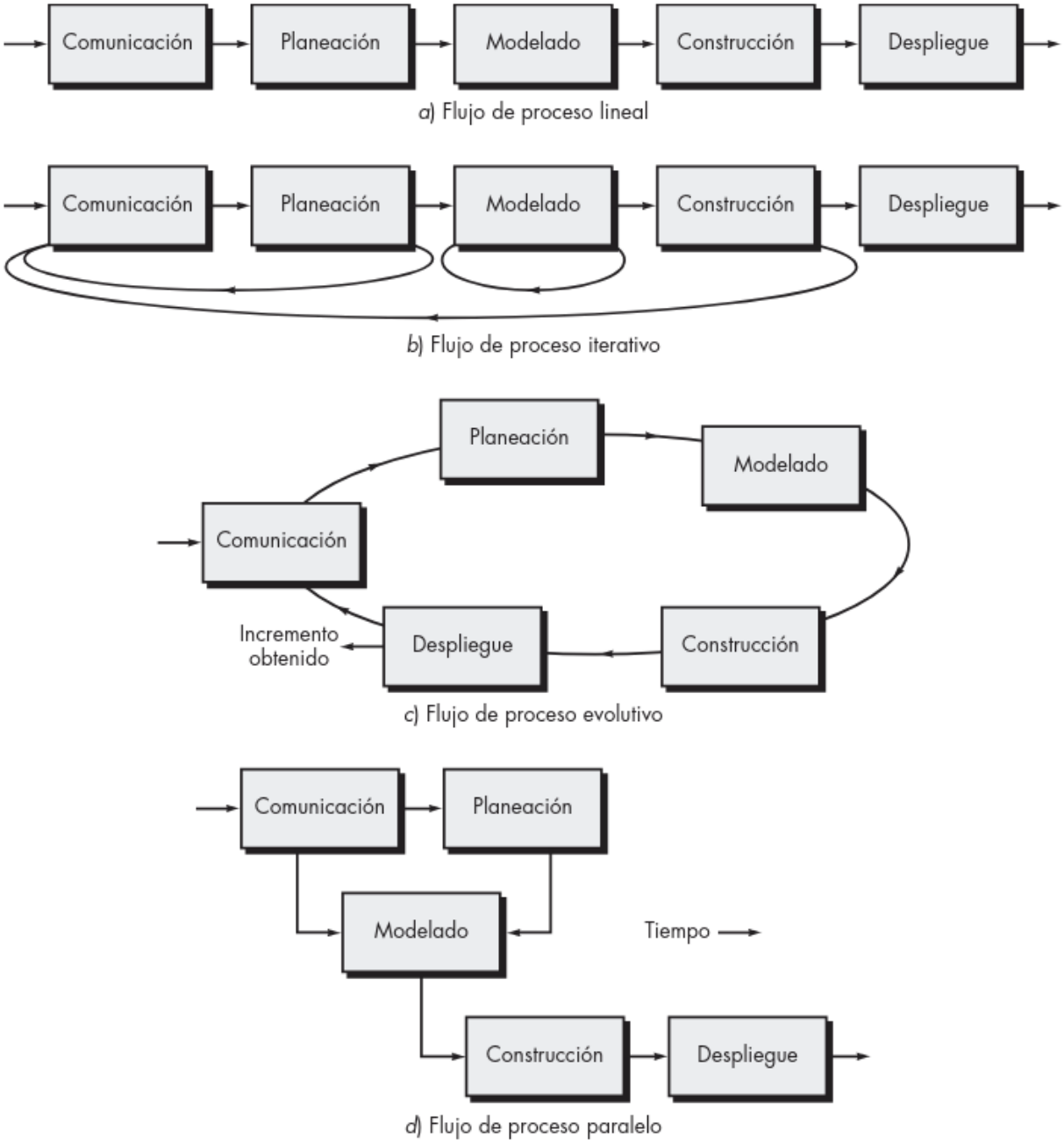

Figure 1: Tipos de flujo

Identificacion de un conjunto de tareas

Debe escogerse el conjunto de tareas que se adapte mejor a las necesidades del proyecto y a las características del equipo. Esto implica que una acción de la ingeniería de software puede adaptarse a las necesidades específicas del proyecto de software y a las características del equipo del proyecto.

Patrones del proceso

Cada equipo se enfrenta a problemas conforme avanza el desarrollo, un **patrón del proceso** describe un problema relacionado con el proceso que se encuentra durante el trabajo de ingeniería de software, identifica el ambiente en el que surge el problema y sugiere una o más soluciones para el mismo.

Amber propuso un formato para describir un patron de proceso:

- Nombre del patron
- Fuerzas
- Tipo(patron de etapa, patron de tarea o patron de fase)
- Contexto inicial
- Problema
- Solucion
- Contexto resultante
- Patrones relacionados
- Usos y ejemplos conocidos

Dan un mecanismo efectivo para enfrentar problemas asociados con cualquier proceso del software. Los patrones permiten desarrollar una descripción jerárquica del proceso, que comienza en un nivel alto de abstracción (un patrón de fase).

Evaluacion y mejora del proyecto

La existencia de un proceso no garantiza que el software se entregue a tiempo o que cumpla con lo pedido. Para evitar esto, el proceso en si puede evaluarse para garantizar que por lo menos cumpla con ciertos criterios que se haya demostrado que son escenciales para el exito.

Algunos enfoques para la evaluacion y mejora son:

- **Metodo de evaluacion del estandar CMMI para el proceso de mejora(SCAMPI)**: Modelo de cinco fases para evaluar el proceso.
- **Evaluacion basada en CMM para la mejora de proceso interno(CBA IPI)**: Tecnica de diagnostico para evaluar madurez relativa.
- **SPICE(ISO/IEC 15504)**: Estandar, define requerimientos para evaluar el proceso. Ayuda a tener una evaluacion objetiva.
- **ISO9001**: Estandar generico que se enfoca en mejorar la calidad de los productos, directamenta aplicable al desarrollo de software.

  Nota: CMMI significa Capability Maturity Model Integration. SEI CMMI y SEI CMM son tecnicas de evaluacion.

Modelos del proceso prescriptivo

Propuestos para poner orden, an resultado ser muy utiles, pero aplicado al software el producto que genera sigue estando "al borde del caos". A continuacion se mustran algunos de estos modelos:

- Modelo de la cascada
- Modelo de proceso incremental
- Modelo de proceso evolutivo(Involucra el modelo espiral)
- Modelos concurrentes

El reto principal es establecer un balance apropiado entre estos parametros criticos del proyecto y el producto, y la satisfaccion del cliente(el que tendra la palabra final con respecto a la calidad).

> **Nota**: Descripcion de estos modelos estan en los apuntes de clase.

Modelo de procesos especializado

Comparten algunaas caracteristicas con los modelos tradicionales, pero tienden a aplicarse cuando se tiene un enfoque muy especifico.

Desarrollo basado en componentes

Incarpora muchas de las caracteristicas del modelo espiral, es de naturaleza evolutiva y demanda enfoque iterativo. Su principal caracteristica es que **contruye aplicaciones a partir de fragmentos de software prefabricados**.

Incorpora las siguientes etapas:

1. Se investiga y evaluan productos ya disponibles.
2. Se consideran aspectos de integracion.
3. Se disena una arquitectura que reciba los coomponentes.
4. Se integran los componentes a la arquitectura.
5. Se efectuan pruebas para asegurar la funcionalidad adecuada.

Este modelo lleva a la reutilizacion del software, reduciendo el tiempo y costo del desarrollo.

Modelo de metodos formales

Tiene un enfoque matematico, desarrollando un sistema por medio de una notacion matematica. Lo ambiguo o incorrecto se corrije con mucha mas facilidad mediante el analisis matematico. Aunque no es el modelo mas seguido, promueve software libre de defectos. Ademas, permite construir software de muy alta calidad en terminos de seguridad o donde los errores son muy costosos.

Sin embargo, tiene sus desvantajas:

- El desarrollo consume mucho tiempo y es caro.
- Se requiere mucha capacitacion del personal.
- Es dificil de implementar con clientes que no tienen la complejidad tecnica.

> **Nota**: Existe una variante de este enfoque que se denomina *ingenieria de software de quirofano*.

Desarrollo orientado a aspectos

Sin importar el proceso elejido, estos tienen varias caracteristicas en comun. A medida que se hacen mas sotisficados los proyectos, el desarrollo de software presenta **preocupaciones globales**. Afectan multiples funciones, caracteristicas e informacion del sistema.

Por lo tanto, el DSOA es un paradigma nuevo que se enfoque en definir, especificar y construir aspectos, estos pueden ser de:

- Persistencia(almacenamiento)
- Seguridad
- Transacciones(control de concurrencia, por ejemplo)

El proceso unificado

La tendencia en el software es hacia sistemas mas grandes y complejos, principalmente a que las computadoras son cada ves mas potentes. El proceso unificado es un intento de obtener mejores caracteristicas de los modelos tradicionales, pero que a su vez implemente los mejores principios del desarrollo **agil**. Sugiere flujo iterativo e incremental, lo que da la sensacion evolutiva que resulta esencial en el desarrollo.

Fases del proceso unificado

Estas faces involucran las **actividades del proceso** mencionadas anteriormente, mostrando como pueden usarse para describir cualquier modelo.

- **Fase de concepcion**: Comunicacion con cliente y planeacion.
- **Fase de elaboracion**: Comunicacion y modelado del modelo general de proceso. Es comun que al finalizar se hagan modificaciones al plan.
- **Fase de construccion**: Actividad de construccion. Desarrolla y adquiere componentes del software(por ejemplo, el lanzamiento). Pruebas unitarias, integracion, etc.
- **Fase de transicion**: Ultimas etavas de la actividad de construccion y primera parte de despliegue general(entrega y feedback). Se da a los usuarios el software de prueba. Se genera informacion de apoyo(manuales, por ejemplo).
- **Fase de produccion**: Coincide con la actividad de despliegue del proceso general. Se reportan defectos y solicitudes.

El **flujo de trabajo** de la ingenieria de softare esta distribuido en todas las fases del PU. Sin embargo, no toda tarea mencionada es necesariamente realizada en todos los proyectos de software(se adapta al equipo).

> **Nota**: Las fases no ocurren en secuencia, sino de forma escalonada.

Modelos del proceso personal y del equipo

El proceso esta cerca de las personas que haran el trabajo, si el modelo se realiza a nival corporativo, sera efiscaz solo si acepta una adaptacion significativa. Idealmente, el proceso debe ajustarse del mejor modo a los requerimientos y cumbrir, al mismo tiempo, las amplias necesidades del equipo y de la organizacion. Alternativamente, el equipo puede ser el que se encargue de realizar el proceso, enfocandoce mas en el equipo y solo cubrir las necesidades generales de la organizacion.

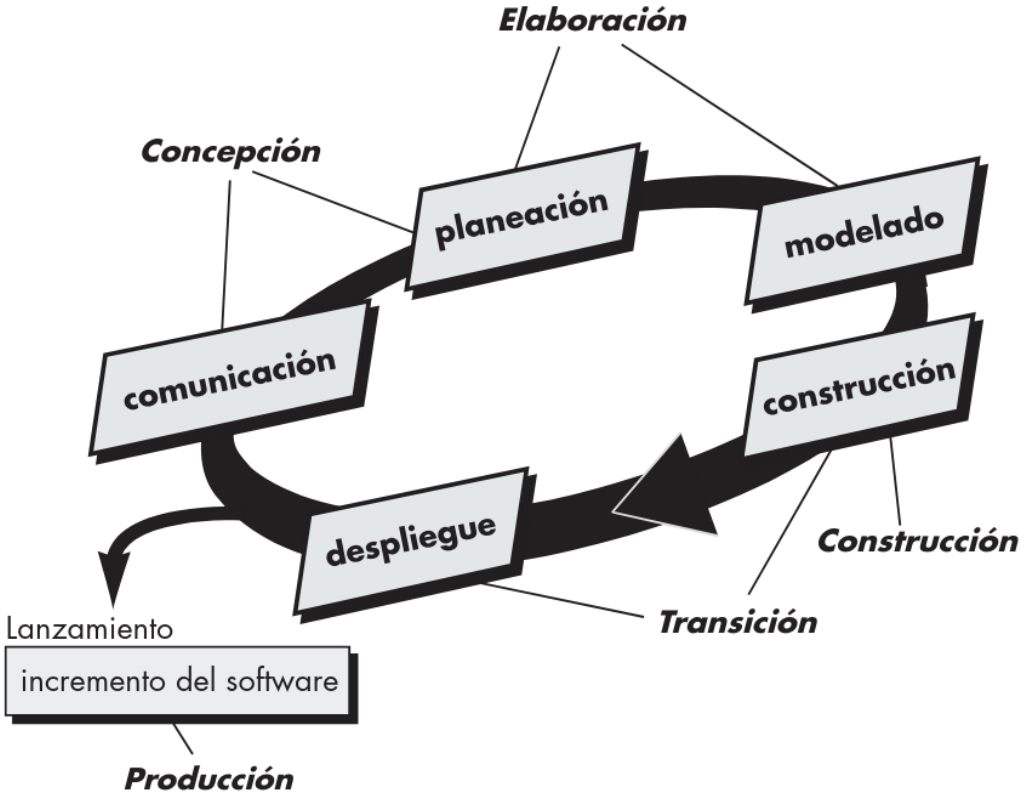

Figure 2: Fases del proceso unificado

Proceso personal del software(PPS)

Se enfoca en la medicion personal tanto del producto del trabajo que se genera como de su calidad, responsabiliza al profesional acerca de la planeacion del proyecto(por ejemplo, programacion de actividades) y delega en el practicante la calidad de los productos. Define cinco actividades estructurales:

- **Planeacion**: Aisla requerimientos y desarrolla estimaciones(numero de defectos, recursos, etc.). Se identifican las tareas de desarrollo y se crea un programa.
- **Diseno de alto nivel**: Especificaciones externas para cada componente, si hay incertidumbre se elaboran prototipos.
- **Revision de diseno de alto nivel**: Se aplican metodos de verificacion formal para descubrir errores. Se miden las tareas y resultados.
- **Desarrollo**: Mejora y revisa el diseno, se genera el codigo, se revisa y prueba. Se miden las tareas y trabajos de importancia.
- **Post mortem**: Se determina la eficacia del proceso por medio de mediciones(nuevas o recolectadas). Se mejora la eficacia.

Se centra en detectar errores lo antes posible, no a sido adoptado con amplitud en la industria debido a que es culturamente dificil realizar el nivel requerido de mediciones.

Proceso del equipo de software(PES)

El objetivo de este es construir un equipo **autodirigido** para el proyecto, que se organice para producir software de alta calidad. Tiene los siguientes objetivos:

- Formar equipos autodirigidos.
- Mostrar a los gerentes como dirigir y motivar a sus equipos.
- Acelerar la mejora del proceso de software(CMM).
- Brindar a las organizacion una guia para la mejora.
- Facilitar la ensenanza universitaria de amplitudes de equipo.

Un equipo autodirigido tiene una comprension consistente de sus metas y objetivos, debe haber colaboracion interna y comunicacion externa. Define las siguientes actividades estructurales(similar a PPS):

- Inicio del proyecto
- Diseno de alto nivel
- Implementacion
- Integracion y pruebas
- Post mortem

Utiliza amplia variedad de **scripts**, los cuales definen formatos y estandares que guian a los miembros del equipo. Reconoce que los mejores equipos de software son los autodirigidos, los miembros son los responsables de seguir con los objetivos y adaptarse en caso de ser necesario.

Tecnologia del proceso

El equipo debe adaptar uno o mas de los modelos de proceso mencionados, para ello existen **herramientas de tecnologia del proceso**. Una vez creado un proceso aceptable, se emplean otras herramientas de tecnologia para asignar, vigilas e incluso controlas todas las actividades. Tambien puede ser usada para

coordinar el empleo de otras herramientas que sean apropiadas para una tarea en particular. Margaret Davis hace comentarios sobre la dualidad del producto y del proceso:

> En tanto que la tendencia natural de un péndulo es alcanzar el estado de reposo en el punto medio entre dos extremos, la atención de la comunidad del software cambia constantemente porque se aplica una nueva fuerza al fallar la última oscilación. Estos vaivenes son dañinos en sí mismos porque confunden al profesional promedio del software al cambiar en forma radical lo que significa hacer el trabajo bien. Los cambios periódicos no resuelven "el problema" porque están predestinados a fallar toda vez que el producto y el proceso son tratados como si fueran una dicotomía en lugar de una dualidad.
>
> Obtenemos sentimientos de satisfacción por la reutilización de nuestros productos, ya sea que lo hagamos nosotros u otras personas.

Como profesional creativo, hay que ser capaz de obtener tanta satisfaccion del proceso como del producto final. La dualidad planteada es un elemento importante para hacer que personas creativas se involucren.

TL;DR

Un modelo general del proceso incluye un conjunto de actividades, cada uno de los modelos puede describirse por un flujo distinto. Los patrones del proceso pueden utilizarse para resolver problemas comunes que surgen durante el proceso. Se dividen en 3 grandes grupos:

- Modelos de proceso prescriptivo
- Modelos de proceso secuencial
- Modelos de proceso icnremental

Capitulo 3: Desarrollo agil

Algunos autores creen que los procesos prescriptivos tienen una gran desventaja y es que:

> Olvidan las flaquezas de las personas cuando construyen software. Los ingenieros de software no son robots. Sus estilos de trabajo varían mucho; tienen diferencias significativas en habilidad, creatividad, orden, consistencia y espontaneida.
>
> Como la consistencia de las acciones es una debilidad humana, las metodología que requieren mucha disciplina son frágiles.

Manifesto agil

- Los **individuos y sus interacciones**, sobre los procesos y las herramientas
- El **software que funciona**, más que la documentación exhaustiva
- La **colaboración con el cliente**, y no tanto la negociación del contrato
- **Responder al cambio**, mejor que apegarse a un plan

Que es la agilidad?

Aun mas especificamente, que es la agilidad en el contexto del desarrollo de software. Es algo mas que una respuesta efectiva al cambio, incluye la filosofia descripta en el manifesto. Pone el énfasis en:

- La entrega rápida de software funcional y resta importancia a los productos intermedios del trabajo (lo que no siempre es bueno).
- Adopta al cliente como parte del equipo de desarrollo y trabaja para eliminar la actitud de "nosotros y ellos" que todavía invade muchos proyectos de software.
- Reconoce que la planeación en un mundo incierto tiene sus límites y que un plan de proyecto debe ser flexible.

Es importante para poder aplicar la agilidad que se disene el proceso de manera que:

- Permita al equipo adaptar las tareas.
- Ejecutar la planeacion de manera fluida.
- Dejar solo los productos del trabajo esenciales.
- Poner enfasis en una estrategia de entrega incremental.

La agilidad y el costo del cambio

Empiricamente te puede observar que el costo de un proyecto evoluciona de forma no lineal a medida que el proyecto avanza. En especial si se requiere ahcer un cambio en una etapa avanzada, donde es muy costoso y lleva mucho tiempo evitar que se haga un cambio sin efectos colaterales no intencionados.

Un proceso agil bien disenado aplana el costo de la curva de cambio(ver imagen).

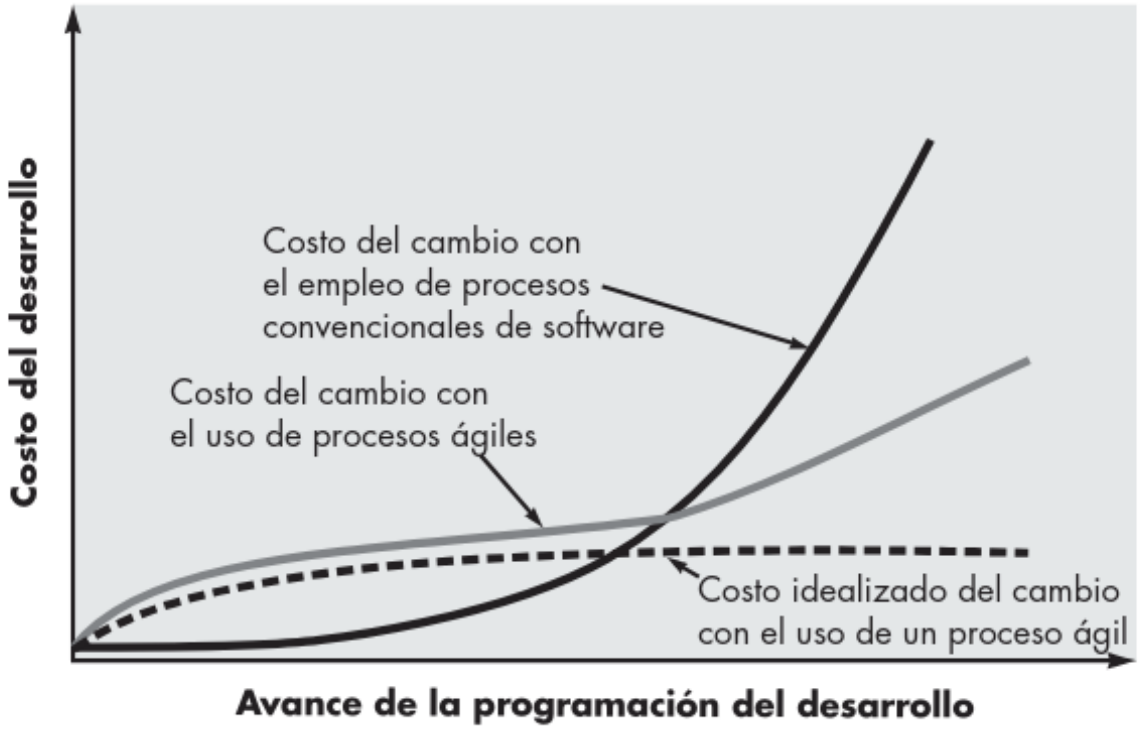

Figure 3: Curva de costo durante un programa de desarrollo

Cuando ésta se acopla con otras prácticas ágiles, como las pruebas unitarias continuas y la programación por parejas (que se estudia más adelante, en este capítulo), el costo de hacer un cambio disminuye.

Que es un proceso agil?

Cualquier proceso del software ágil se caracteriza por la forma en la que aborda cierto número de suposiciones clave:

- Es dificil predecir como cambiaran los requisitos.
- El diseno y la construccion deben ejecutarse, generalmente, de forma simultanea.
- El análisis, el diseño, la construcción y las pruebas no son tan predecibles como nos gustaría.

Un proceso agil debe:

- Adaptarse incrementalmente(con feedback del cliente).
- Instuirse una estrategia de desarrollo incremental.
- Entregar incrementos de software(prototipos o porciones).

Principios de agilidad

Se definen 12 principios de agilidad para aquellos que la quieran alcanzar:

1. La prioridad más alta es satisfacer al cliente a través de la entrega pronta y continua de software valioso.
2. Son bienvenidos los requerimientos cambiantes, aun en una etapa avanzada del desarrollo.
3. Entregar con frecuencia software que funcione, de dos semanas a un par de meses, de preferencia lo más pronto que se pueda.
4. Las personas de negocios y los desarrolladores deben trabajar juntos, a diario y durante todo el proyecto.
5. Hay que desarrollar los proyectos con individuos motivados. Debe darse a éstos el ambiente y el apoyo que necesiten, y confiar en que harán el trabajo.
6. El método más eficiente y eficaz para transmitir información a los integrantes de un equipo de desarrollo, y entre éstos, es la conversación cara a cara.
7. La medida principal de avance es el software que funciona.
8. Los procesos ágiles promueven el desarrollo sostenible. Los patrocinadores, desarrolladores y usuarios deben poder mantener un ritmo constante en forma indefinida.
9. La atención continua a la excelencia técnica y el buen diseño mejora la agilidad.
10. Es esencial la simplicidad: el arte de maximizar la cantidad de trabajo no realizado.
11. Las mejores arquitecturas, requerimientos y diseños surgen de los equipos con organización propia.
12. El equipo reflexiona a intervalos regulares sobre cómo ser más eficaz, para después afinar y ajustar su comportamiento en consecuencia.

No todos los modelos agil aplican los 12 principios, pero definen la idea principal y la filosofia de agil.

La politica de desarrollo agil

Hay mucho debate hacerca de la efectividad de agil en comparacion con los metodos tradicionales. Esto se torno en un tema de debate sencible que rapidamente puede llevar a una discusion. Sin embargo, la verdadera pregunta es cual es la mejor forma de implementar agil?

No hay respuestas absolutas a esto, entonces, hay mucho por ganar si se considera lo mejor de ambas escuelas, y virtualmente no se gana nada si se denigra cualquiera de los enfoques.

Factores humanos

Los defensores de agil se enfocan principalmente en los **factores personales**. Si los miembros del equipo de software son los que van a generar las características del proceso que van a aplicarse a la elaboración de software, entre ellos debe existir cierto número de características clave, mismas que debe compartir el equipo ágil como tal:

- Competencia
- Enfoque comun
- Colaboracion
- Habilidad para tomar decisiones
- Capacidad para resolver problemas difusos
- COnfianza y respeto mutuos
- Organizacion propia

Programacion extrema(XP)

Es el enfoque mas utilizado en el desarrollo agil, a continuacion se dara un panorama.

Valores

Se define un conjunto de 5 valores:

- Comunicacion
- Simplicidad
- Retroalimentacion
- Valentia
- Respeto

Proceso

Usa un enfoque orientado a objetos como paradigma preferido. Las actividades estructurakes clave son:

- Planeamiento

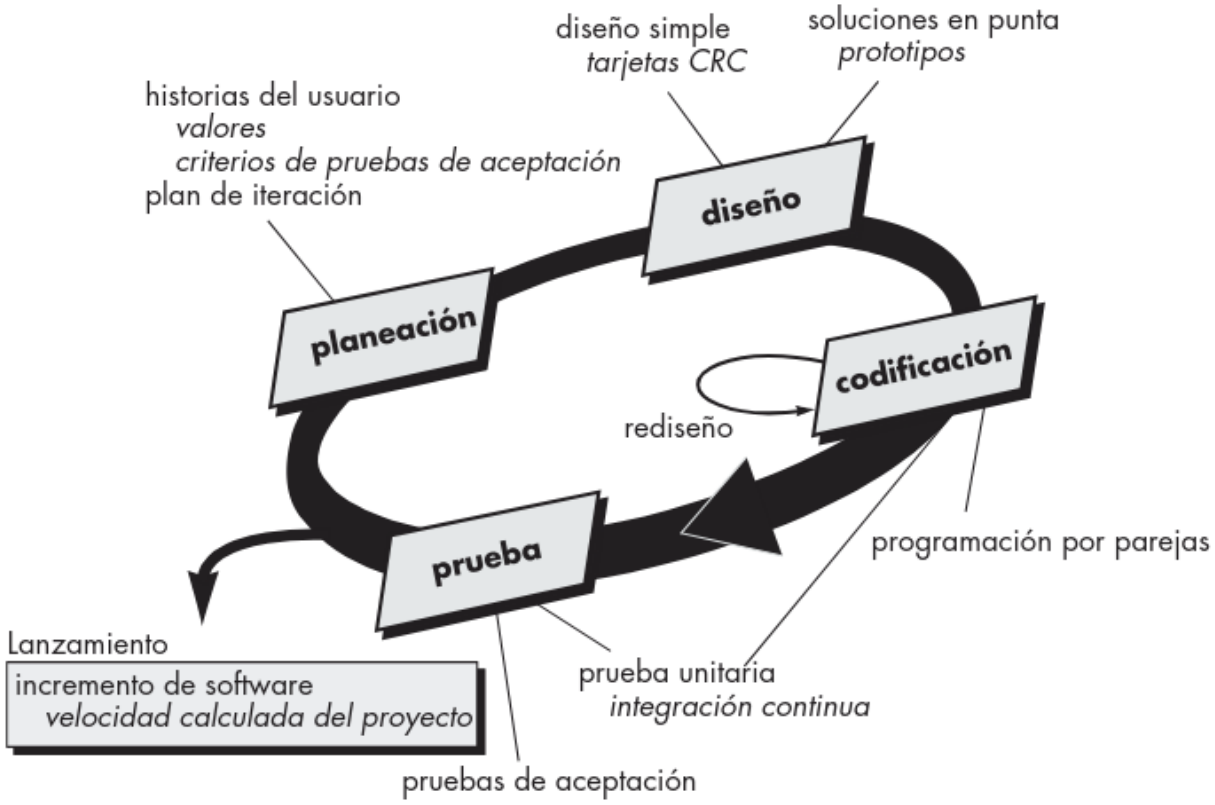

Figure 4: Actividades de XP